



ADVENTIST UNIVERSITY OF CENTRAL AFRICA
FACULTY OF INFORMATION TECHNOLOGY
MASTER'S IN BIG DATA ANALYTICS

Online Retail: Segmenting a Real Storefront

*Unsupervised and Supervised Machine Learning Applied to the
UCI Online Retail II Transaction Dataset*

FINAL GROUP PROJECT

Machine Learning on Real, Public Data

Lecturer

Dr. Lema LOGAMOU SEKNEWNA

Submitted by

Group 3

#	Student ID	Full Name
1	20251MBI016	Mudahogora Justine
2	20251MBI019	Niyomufasha Emerance
3	20251MBI022	Ngabonziza Nestor
4	20251MBI054	Arthur Butera
5	20251MBI008	Nsabimana Jean Paul

Dataset: Online Retail II (UCI Machine Learning Repository)
<https://archive.ics.uci.edu/dataset/502/online+retail+ii>

September 16, 2026

Contents

Executive Summary	2
1 Introduction & Problem Framing	2
2 Data Description & Cleaning	2
2.1 Cleaning It Up	2
3 Exploratory Data Analysis	3
3.1 The Basics	3
3.2 Most Purchases Are Small; a Few Are Huge	3
3.3 Sales Follow the Calendar	3
3.4 What Sells	4
3.5 Where the Money Comes From	4
3.6 A Small Number of Customers Matter a Lot	5
3.7 Cancellations Are Not Rare	5
3.8 Quantity Drives Revenue, Not Price	5
4 Unsupervised Analysis: Customer Segmentation	6
4.1 Method: RFM Features and K-Means Clustering	6
4.2 Mathematical Details	6
4.3 The Five Segments	8
4.4 Business Implications	9
5 Supervised Modelling: Predicting Customer Churn	9
5.1 Defining the Prediction Problem	9
5.2 Target and Regressor Variables	9
5.3 Checking for Multicollinearity	10
5.4 Models Compared	11
5.5 How Each Model Works	11
5.6 Model Performance	12
5.7 Checking for Overfitting	13
5.8 Choosing the Best Model	14
5.9 Confusion Matrix Analysis	14
6 Model Interpretation: What Predicts Churn	15
7 Discussion: Limitations, Ethics, and Future Work	16
7.1 Limitations	16
7.2 An Ethical or Bias Risk Worth Naming	16
7.3 What We Would Do With More Time	16
8 Conclusion & Recommendations	17
References	17
Appendix: How the Customer Explorer App Works	17

Executive Summary

Prepared for: the CRM/Retention team.

You are sitting on two years of transaction history and, until now, have been treating every customer the same way. That is costing you money on two fronts: you are spending marketing budget on customers who will never be worth much, and you are not noticing your best customers slipping away until they are already gone.

We split your customer base into five plain-language groups. One of them, **VIP**, is just 15% of your customers but generates **70% of your total revenue**. Protecting that group is the single highest-leverage thing your team can do. At the other end, **One-Time Buyers** and customers we call **Lost/Let Go** together make up over 40% of your customer base but contribute less than 5% of revenue combined money spent chasing them is largely money wasted.

We also built an early-warning model that flags, six months in advance, which currently active customers are likely to stop buying. It correctly catches about 73% of customers who go on to churn, using only signals already in your order history (how recently, how often, and how much someone buys). The strongest single warning sign, by a wide margin, is how long it has been since a customer's last order.

Our recommendation: route VIP and Loyal customers into a retention/loyalty track, route customers the model flags as high-risk into a targeted win-back campaign before they go quiet for good, and stop spending marketing budget on One-Time Buyers and the Lost/Let Go segment beyond the cheapest possible touch. The sections that follow walk through the data, the method, and the numbers behind this recommendation.

1 Introduction & Problem Framing

A CRM team staring at a spreadsheet of over a million transaction lines needs to know: who's already valuable and worth keeping happy, who used to be valuable but has gone quiet and is worth winning back, and who's realistically never coming back, so money stops being wasted chasing them.

We answer this in two steps. First, we group customers into segments based on past behavior **unsupervised learning**, since we're not predicting anything, just describing who is who. Second, we build a model that predicts which customers are likely to churn, **supervised learning**, trained on customers whose fate we already know, to make a call on customers whose fate we don't know yet.

Both steps rely on three simple ideas about a customer, known as **RFM: Recency** (days since their last purchase), **Frequency** (how many separate orders they've placed), and **Monetary** (how much they've spent in total) three numbers that turn out to say a surprising amount about a customer.

2 Data Description & Cleaning

We used the **Online Retail II** dataset, publicly available from the UCI Machine Learning Repository: every transaction line from a UK-based online retailer selling gifts and homeware, from December 2009 to December 2011. In its raw form the dataset has **1,067,371 rows**, and it is messy, the way real business data usually is.

2.1 Cleaning It Up

Before we could trust any statistic from this data, we had to fix it. Here is what we did, step by step, and why.

What we did	Rows removed	Rows left
Started with		1,067,371
Fixed data types (dates, text fields)	0	1,067,371
Removed exact duplicate rows	12,133	1,055,238
Dropped rows with no Customer ID	242,870	812,368
Dropped rows with no product description	0	812,368
Removed non-product codes (postage, fees, test entries)	3,706	808,662
Set aside cancelled orders separately	17,879	790,783
Removed rows with zero or negative quantity or price	62	790,721
What we ended up with		790,721 rows

Table 1: Every cleaning step, and how many rows it touched. Nothing was removed without a reason we could point to.

A few of these calls are worth explaining. We dropped rows with no Customer ID rather than guess one, since every downstream analysis is about a specific customer. We kept cancelled orders, just set them aside in their own table, since a cancelled order isn't really a sale but is still worth having for a future returns analysis. And we stripped out postage, bank fees, and test codes, since those aren't real products and would have quietly inflated some customers' purchase counts.

After all this, the clean dataset had **zero missing values** left, and every number in this report is built on top of it.

3 Exploratory Data Analysis

Before modeling anything, we spent time just looking at the data: what is typical, what is extreme, what patterns jump out.

3.1 The Basics

The clean dataset covers **738 days** (Dec 2009 to Dec 2011), with **5,852 unique customers**, **36,594 orders**, and **4,621 different products**, sold across **41 countries**.

3.2 Most Purchases Are Small; a Few Are Huge

Quantity, price, and revenue per item are not evenly distributed. Most purchases are small, but a few purchases are extremely large. The typical order line is worth about £12, while the largest single item was worth more than £168,000. This difference is important because a few very large values could strongly affect the analysis. Therefore, some of these variables are later transformed using logarithms before clustering, so that very large values do not dominate the results.

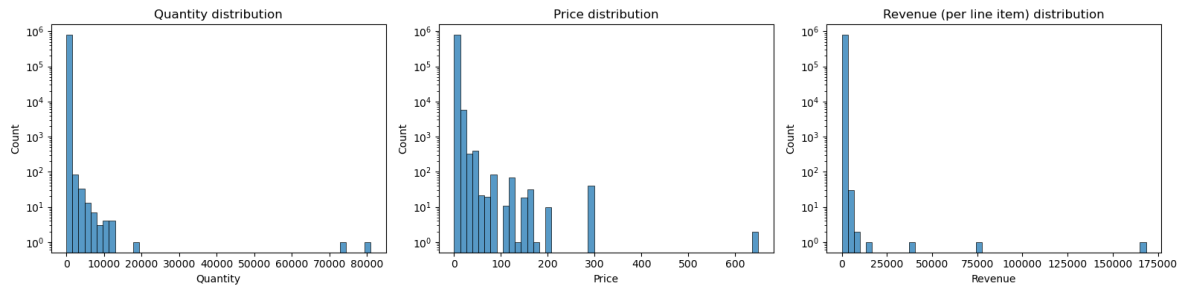


Figure 1: Quantity, price, and revenue per line item, all skewed heavily toward small values, with a long tail of large ones.

3.3 Sales Follow the Calendar

Revenue and order volume both spike hard in the run-up to Christmas, in both years. That is not surprising for a gift retailer.

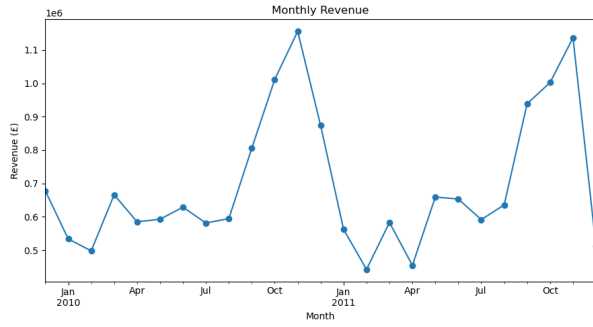


Figure 2: Monthly revenue. Clear seasonal peaks before Christmas.

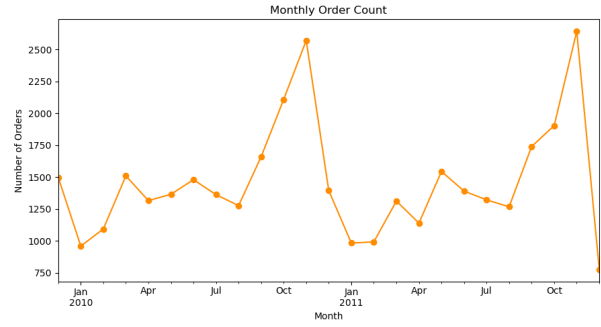


Figure 3: Monthly order count, following the same seasonal shape as revenue.

Ordering also concentrates on weekday business hours (Figure 4), which hints that a good number of these customers may be buying for resale or business use, not just personal gifts.

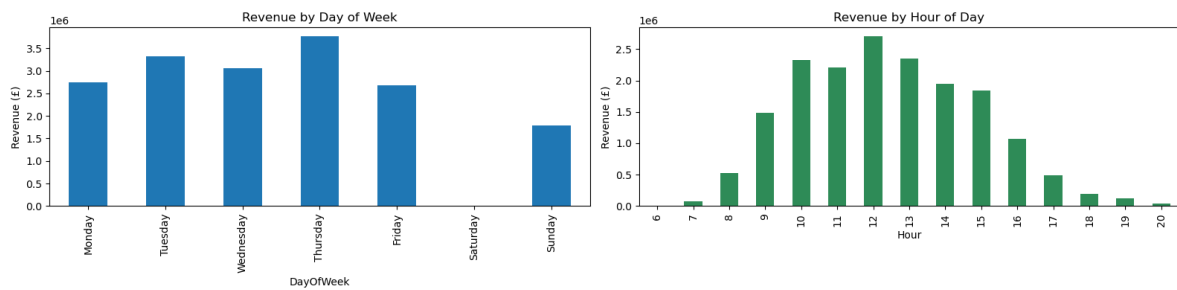


Figure 4: Revenue by day of week and hour of day.

3.4 What Sells

The single best-selling product by revenue was the *Regency Cakestand 3 Tier*, bringing in over £285,000 on its own. The *White Hanging Heart T-Light Holder* shows up as a top seller by both quantity and revenue, a sign that it is a dependable staple rather than a one-off seasonal hit.

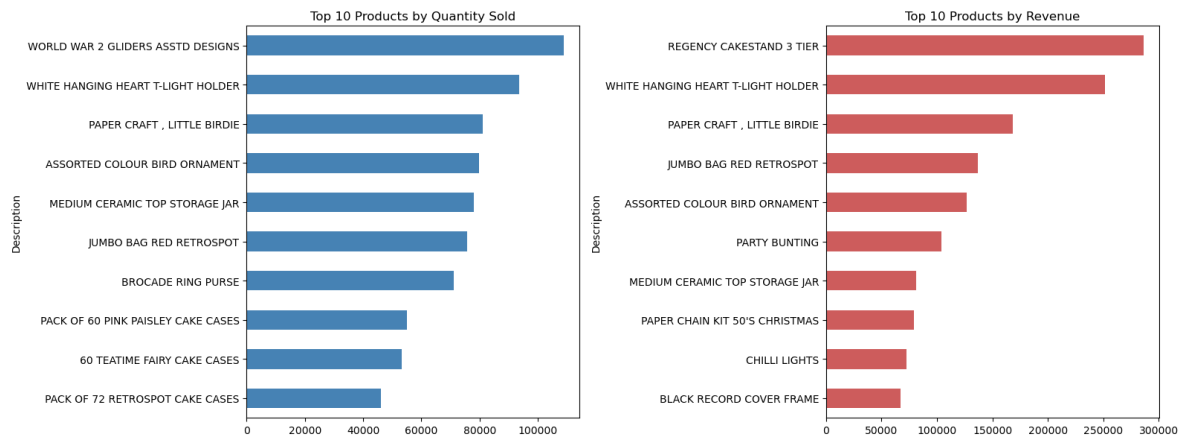


Figure 5: Top 10 products by quantity sold (left) and revenue (right).

3.5 Where the Money Comes From

The UK is, by far, the main market: **83.8% of all revenue** and 33,361 of 36,594 orders. Everywhere else, including Ireland, the Netherlands, Germany, and France, is a distant second. This matters as a caveat: our customer segments and churn model are, in practice, mostly describing UK shopping behavior, since that is most of the data.

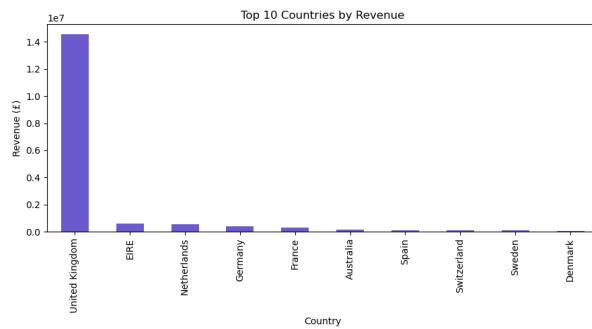


Figure 6: Top 10 countries by revenue (UK dominates at 83.8%).

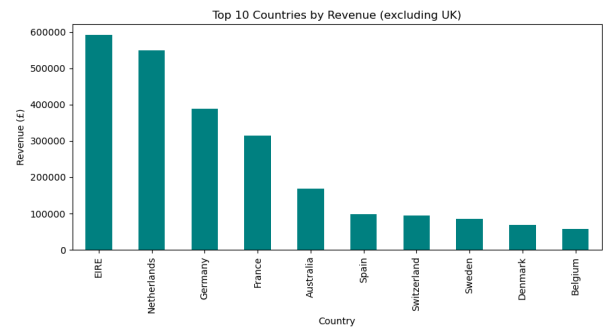


Figure 7: Top 10 countries by revenue, UK excluded, to see the rest of the picture.

3.6 A Small Number of Customers Matter a Lot

This is probably the single most useful fact in the whole EDA: **23% of customers generate 80% of total revenue** (1,348 out of 5,852 people). Most customers order rarely. The typical, or median, customer has placed just 3 orders, while a small handful order constantly. The most active customer placed 373 orders.

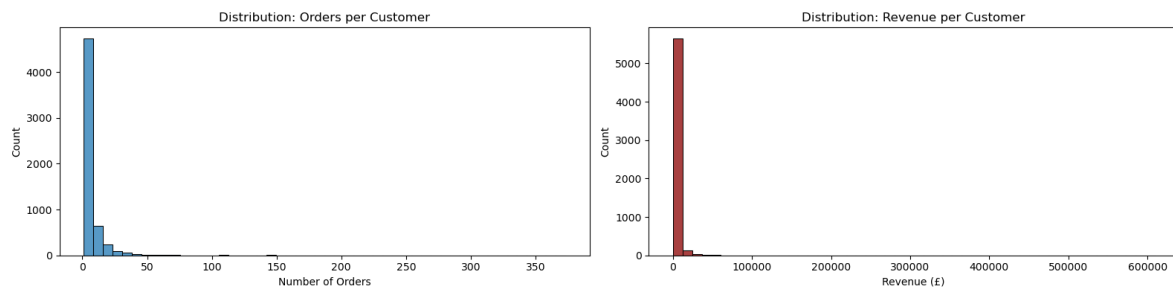


Figure 8: How many orders customers place (left) and how much they spend (right), both heavily concentrated among a few.

This one number is really the whole business case for segmentation. Treating every customer the same way, with the same marketing spend, makes no sense when the base is this lopsided.

3.7 Cancellations Are Not Rare

About **1 in 5 orders gets cancelled**. There were 7,283 cancelled invoices against 36,594 completed sales invoices, which works out to 19.9% of the sales invoice count. Cancellations are, like everything else, concentrated in the UK. This would be worth its own follow-up analysis someday, but was not the focus here.

3.8 Quantity Drives Revenue, Not Price

Quantity and revenue move together closely ($r = 0.83$), while price barely correlates with either. In plain terms, people spend more here mostly by buying more items, not by buying pricier ones.



Figure 9: Correlation between quantity, price, and revenue.

4 Unsupervised Analysis: Customer Segmentation

Now that we understand the data, the first real question is whether we can put customers into a small number of groups that actually mean something to a CRM team.

4.1 Method: RFM Features and K-Means Clustering

For each of the 5,852 customers, we computed their Recency, Frequency, and Monetary numbers. Because Frequency and Monetary are so skewed (a few whale customers spend enormously more than everyone else), we squashed them with a log transform before comparing customers to each other. Otherwise those few big spenders would swamp the whole grouping and everyone else would look the same.

We then tried grouping customers into anywhere from 3 to 8 clusters using K-Means, scoring each split with the silhouette score. Purely on that score, 3 clusters won (silhouette ≈ 0.40), but three groups are too coarse to tell a VIP apart from a one-time buyer, which is exactly what the CRM team needs. So we took the best-scoring split with at least 4 clusters, which was $k = 5$ (silhouette = 0.366). Each cluster was then named using plain RFM-quintile logic (for example, “recent, frequent, big spender” becomes “VIP”), and each of the five statistical clusters turned out to correspond to a distinct, business-meaningful customer type, so we kept all **5 clusters as 5 named segments: VIP, Loyal Customer, At Risk, One-Time Buyer, and Lost/Let Go**.

4.2 Mathematical Details

It is worth spelling out exactly what “Recency, Frequency, Monetary” and “K-Means” mean in numbers, since these formulas are what the rest of the report is built on.

RFM features. For a chosen snapshot date T and a customer c , the three raw features are:

$$\text{Recency}(c) = T - \max\{t : t \text{ is an invoice date of } c\} \quad (\text{days}) \quad (1)$$

$$\text{Frequency}(c) = |\{\text{distinct invoices placed by } c\}| \quad (2)$$

$$\text{Monetary}(c) = \sum_{i \in \text{orders of } c} \text{Quantity}_i \times \text{Price}_i \quad (3)$$

Because Frequency and Monetary are heavily right-skewed, both were log-transformed before clustering:

$$x' = \log(1 + x) \quad (4)$$

and then standardized to zero mean and unit variance,

$$z = \frac{x' - \mu}{\sigma}, \quad (5)$$

so that no single feature dominates the distance calculation just because it happens to be measured on a larger numeric scale. We use these three specific numbers, rather than folding everything into one "value" score, because each answers a different question: Recency asks if someone is still around, Frequency asks if buying here is a habit, and Monetary asks how much they're worth – a customer can score high on one and low on another.

K-Means clustering. Given n customers represented as points $z_1, \dots, z_n \in \mathbb{R}^3$ (standardized Recency, Frequency, Monetary), K-Means partitions them into k clusters $C_1, \dots, C_k$ by minimizing the total within-cluster squared distance to each cluster's centroid μ_j :

$$\arg \min_{C_1, \dots, C_k} \sum_{j=1}^k \sum_{z_i \in C_j} \|z_i - \mu_j\|^2, \quad \mu_j = \frac{1}{|C_j|} \sum_{z_i \in C_j} z_i. \quad (6)$$

In plain terms, each customer is assigned to whichever cluster centroid they are closest to, and each centroid is then recomputed as the average of the customers currently assigned to it. This repeats until the assignments stop changing. We chose $k = 5$ using the silhouette score, which for a point i compares its average distance to points in its own cluster (a_i) against its average distance to points in the nearest other cluster (b_i):

$$s_i = \frac{b_i - a_i}{\max(a_i, b_i)}, \quad s_i \in [-1, 1], \quad (7)$$

with a higher average s_i across all customers indicating tighter, better-separated clusters. In our search over $k = 3$ to 8 (Figure 10), the score is highest at $k = 3$ (0.402) and then dips, with a small local peak at $k = 5$ (0.366) before declining steadily at higher k . Inertia falls smoothly and monotonically as k increases, with no sharp elbow, so the silhouette score, plus the business need for at least four actionable groups, is what settles the choice at $k = 5$.

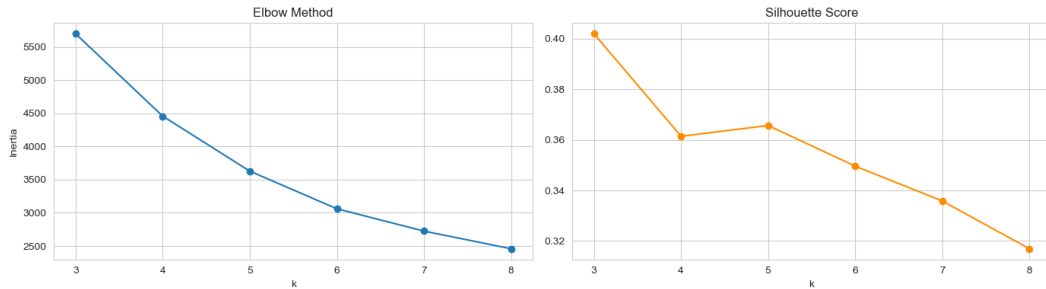


Figure 10: Inertia (elbow method, left) and silhouette score (right) for $k = 3$ to 8. The silhouette is highest at $k = 3$ but we picked $k = 5$ (silhouette = 0.366), the best score among splits fine-grained enough for the CRM team to act on.

PCA for visualization. To plot the clusters in two dimensions, Principal Component Analysis re-expresses the standardized RFM feature space along the directions of greatest variance. If Z is the $n \times 3$ matrix of standardized features with covariance matrix $\Sigma = \frac{1}{n} Z^\top Z$, PCA finds the eigenvectors v_1, v_2, v_3 and eigenvalues $\lambda_1 \geq \lambda_2 \geq \lambda_3$ of Σ , and projects each customer onto the top two components:

$$\text{PC1} = Zv_1, \quad \text{PC2} = Zv_2. \quad (8)$$

The share of total variation each component explains is $\lambda_j / \sum_j \lambda_j$. In our case PC1 explains 74.6% of the variance and PC2 another 20.5%, so the two together capture 95.1% of everything that

distinguishes customers in the RFM space. That is why the two-dimensional scatter in Figure 11 is a trustworthy picture of the clustering, not a distorted one.



Figure 11: Customer segments plotted on the first two principal components of the RFM space (PC1: 74.6% of variance, PC2: 20.5%). The five groups sit in clearly separate regions, ordered along PC1 from Lost/Let Go and One-Time Buyers on the left, through At Risk and Loyal Customers, to VIPs on the right.

4.3 The Five Segments

Segment	Customers	% of Base	Avg. Days Since Last Order	Avg. Orders	Avg. Spend (£)	% of Revenue
VIP	900	15.4%	42	22.8	13,537	70.1%
Loyal Customer	1,692	28.9%	64	5.7	1,941	18.9%
At Risk	840	14.4%	394	3.4	1,291	6.2%
One-Time Buyer	1,340	22.9%	99	1.7	412	3.2%
Lost/Let Go	1,080	18.5%	521	1.2	254	1.6%

Table 2: The five customer segments and what they look like in practice.

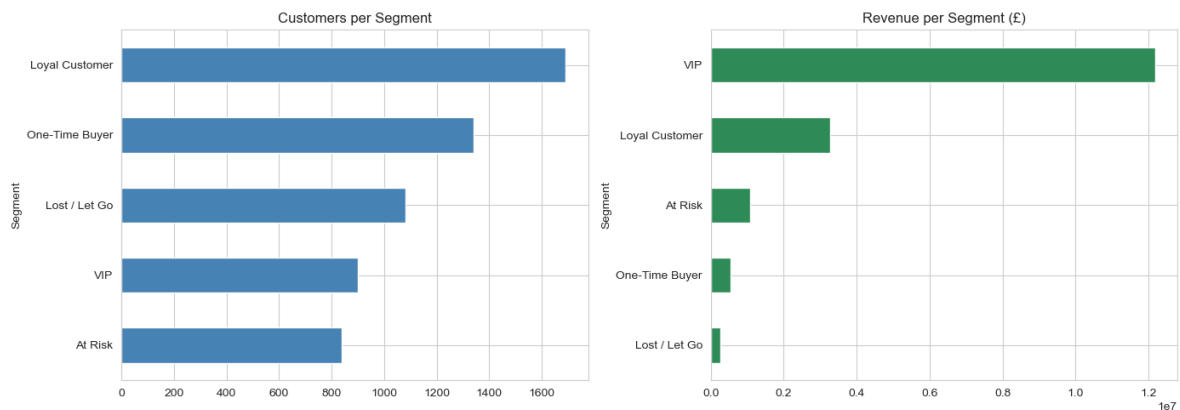


Figure 12: How many customers are in each segment, and how much revenue each one is worth. A small group, the VIPs, carries most of the business.

4.4 Business Implications

The headline: **VIPs are just 15% of customers but bring in over 70% of the revenue.** Meanwhile, One-Time Buyers (22.9% of customers) and the Lost/Let Go segment (18.5%) together make up over **41% of the customer base** but contribute barely **4.8% of revenue** combined. That is a big enough gap that spending marketing money the same way across all five groups would clearly be a mistake. Here is what we would suggest for each group.

- **VIP:** these are the people to protect. White-glove service, early access to new products, loyalty perks.
- **Loyal Customer:** steady repeat buyers. Good targets for upsells, cross-sells, and referral offers.
- **At Risk:** these people used to buy often but have gone quiet, over a year, on average, since their last order. Worth a targeted discount or a re-engagement email before they are gone for good.
- **One-Time Buyer:** bought exactly once, around three months ago on average – still recent enough that a well-timed second-purchase incentive has a reasonable chance of converting them into repeat customers.
- **Lost/Let Go:** bought once or twice, well over a year ago on average, with the lowest average spend of any group. These customers should largely be excluded from paid retention spend; a low-cost, automated win-back email is the most this group is worth pursuing.

This kind of grouping tells us who customers are right now. What it cannot tell us is who is about to change categories, which is exactly the gap the next section tries to close.

5 Supervised Modelling: Predicting Customer Churn

Segmentation is a snapshot. It is useful, but it is backward-looking: it tells you who has already gone quiet, not who is about to. So the second half of this project builds a model that predicts churn before it happens, giving the CRM team a head start.

5.1 Defining the Prediction Problem

We picked a cutoff date and asked: for customers who were active before that date, did they buy anything again in the following 6 months? If not, we call them **churned**. Crucially, every feature the model sees is built only from data before the cutoff. The model never gets to peek at the future it is trying to predict.

- Customers active before the cutoff: **4,952**
- Of those, still active after the cutoff: **3,472**
- Churn rate: **48.1%**, close to a 50/50 split, which is convenient because it means we do not have to do anything special to handle an imbalanced dataset.

We gave the model nine behavior-based signals to learn from, all built from pre-cutoff history: Recency, Frequency, Monetary, Tenure (how long they have been a customer), number of distinct products bought, total items bought, average order value, average items per order, and purchase rate.

5.2 Target and Regressor Variables

Before describing the models, it is worth being precise about exactly what they were asked to predict, and from what.

Target variable. The model predicts a single binary outcome per customer, *Churned*, defined relative to the cutoff date T_c and a 6-month outcome window $[T_c, T_c + 180 \text{ days}]$:

$$y_c = \begin{cases} 1 & \text{if customer } c \text{ placed no order in } [T_c, T_c + 180 \text{ days}] \quad (\text{churned}) \\ 0 & \text{if customer } c \text{ placed at least one order in that window} \quad (\text{retained}) \end{cases} \quad (9)$$

This makes churn prediction a **binary classification** problem: $y_c \in \{0, 1\}$, and each model outputs an estimated probability $\hat{p}_c = P(y_c = 1 \mid \mathbf{x}_c)$ that customer c churns, which is then thresholded, by default at 0.5, to produce a Retained or Churned prediction.

Regressor (feature) variables. All nine predictors $\mathbf{x}_c = (x_1, \dots, x_9)$ are built only from each customer’s transaction history strictly before T_c , so the model never sees information from the outcome window it is trying to predict.

Variable	Definition
Recency	Days between T_c and the customer’s most recent pre-cutoff order.
Frequency	Number of distinct invoices placed before T_c .
Monetary	Total revenue ($\sum \text{Quantity} \times \text{Price}$) from pre-cutoff orders.
Tenure	Days between T_c and the customer’s very first order (i.e. how long they have been a customer).
UniqueProducts	Number of distinct products (StockCode) purchased before T_c .
TotalItems	Total unit quantity purchased before T_c , summed across all orders.
AvgOrderValue	Monetary / Frequency: the typical amount spent per order.
AvgItemsPerOrder	TotalItems / Frequency: the typical number of units per order.
PurchaseRate	Frequency / Tenure: how often, on average, the customer orders per day since becoming a customer.

Table 3: The nine features used to predict churn, all computed from pre-cutoff data only. Each variable was chosen for a reason. Features were standardized for Logistic Regression, but left raw for the tree models, since trees are unaffected by scale.

5.3 Checking for Multicollinearity

Before trusting the model coefficients, it’s worth checking whether any of the nine features are basically measuring the same thing this matters most for Logistic Regression, since strongly correlated features can make individual coefficients unstable even when overall predictions stay fine.

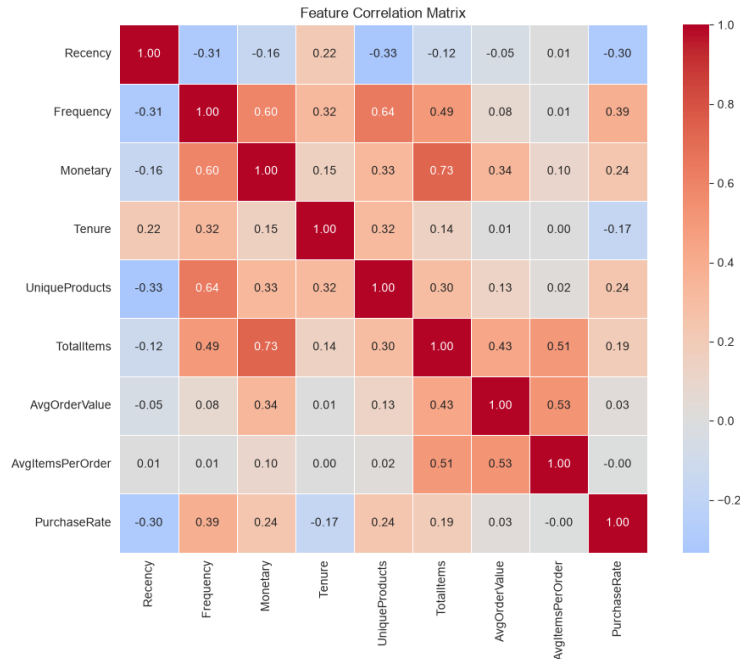


Figure 13: Correlation between every pair of the nine churn features.

The strongest relationships are Monetary with TotalItems ($r = 0.73$), UniqueProducts with Frequency ($r = 0.64$), and Frequency with Monetary ($r = 0.60$) all intuitive, since heavy buyers

tend to spend more and shop across more product lines. None cross the $r = 0.8$ red-flag threshold, so all nine features were kept for every model.

5.4 Models Compared

We trained three models that think about the problem in different ways, on purpose.

- **Logistic Regression:** the simplest option. It is a straight line through the data, easy to explain: each feature gets a weight, and you can say plainly that more recent activity lowers churn risk by a certain percentage. We use it as a baseline and a sanity check.
- **Random Forest:** a collection of decision trees that can pick up on combinations of factors a straight line would miss, for example "frequent and recent" mattering more than either alone.
- **XGBoost:** a more refined tree-based model that builds trees one at a time, each one fixing the mistakes of the last. It is usually the strongest option for this kind of tabular data, so it sets a useful ceiling to compare against.

Running all three side by side tells us not just which one wins, but why. If the fancier models barely beat the simple one, the pattern in the data is probably mostly straightforward. If they win by a lot, there is real complexity a straight line cannot capture.

5.5 How Each Model Works

Each model turns the nine features $\mathbf{x}_c$ into a churn probability in a fundamentally different way.

Logistic Regression. A linear combination of the standardized features is passed through the sigmoid function to squeeze it into a valid probability:

$$\hat{p}_c = \sigma(\mathbf{w}^\top \mathbf{z}_c + b) = \frac{1}{1 + e^{-(\mathbf{w}^\top \mathbf{z}_c + b)}}, \quad (10)$$

which is equivalent to saying the log-odds of churning is a straight-line function of the features:

$$\log\left(\frac{\hat{p}_c}{1 - \hat{p}_c}\right) = \mathbf{w}^\top \mathbf{z}_c + b = w_1 z_1 + w_2 z_2 + \cdots + w_9 z_9 + b. \quad (11)$$

The weights $\mathbf{w}$ are found by minimizing the regularized log-loss over all training customers,

$$\mathcal{L}(\mathbf{w}, b) = -\frac{1}{n} \sum_{c=1}^n \left[y_c \log \hat{p}_c + (1 - y_c) \log(1 - \hat{p}_c) \right] + \lambda \|\mathbf{w}\|_2^2, \quad (12)$$

and each e^{w_j} can be read directly as an odds ratio: how much a one-standard-deviation increase in feature j multiplies a customer's odds of churning, holding the other features fixed. This is exactly how Recency's $2.4\times$ effect in Section 6 is calculated.

Random Forest. A Random Forest builds B independent decision trees, each trained on a bootstrap resample of the customers and a random subset of features at each split, and averages their votes:

$$\hat{p}_c = \frac{1}{B} \sum_{b=1}^B T_b(\mathbf{x}_c), \quad (13)$$

where $T_b(\mathbf{x}_c) \in \{0, 1\}$ is the churn prediction of the b -th tree. Each individual tree grows by repeatedly picking the feature and threshold that most reduces Gini impurity in the resulting child nodes,

$$G(D) = 1 - \sum_{k \in \{0,1\}} p_k^2, \quad p_k = \text{proportion of class } k \text{ in node } D, \quad (14)$$

choosing the split that maximizes the impurity reduction $G(D_{\text{parent}}) - [\frac{|D_L|}{|D|}G(D_L) + \frac{|D_R|}{|D|}G(D_R)]$. Averaging many such trees, each seeing slightly different data and features, is what lets the forest capture non-linear interactions, such as Recency and Frequency mattering together, without any single tree overfitting too badly.

XGBoost. Gradient boosting also builds an ensemble of trees, but sequentially: each new tree f_m is trained to correct the errors of all the trees built so far, rather than being trained independently.

$$\hat{p}_c^{(m)} = \sigma \left(\sum_{t=1}^m f_t(\mathbf{x}_c) \right), \quad f_t \in \mathcal{F} \text{ (regression trees)}. \quad (15)$$

At each boosting round m , XGBoost adds the tree f_m that most reduces a regularized objective combining the log-loss with a penalty on tree complexity:

$$\mathcal{J}^{(m)} = \sum_{c=1}^n \ell(y_c, \hat{p}_c^{(m-1)} + f_m(\mathbf{x}_c)) + \Omega(f_m), \quad \Omega(f) = \gamma T + \frac{1}{2} \lambda \sum_{j=1}^T w_j^2, \quad (16)$$

where T is the number of leaves in the new tree, w_j are the leaf output values, and γ and λ penalize adding leaves and large leaf values respectively. These are the `gamma` and `reg_lambda` settings we tuned in this section. This is why XGBoost tends to generalize a bit more predictably than an unregularized tree ensemble, even though, as Section 5.7 shows, it still overfit somewhat more than Logistic Regression on this dataset.

5.6 Model Performance

Model	Accuracy	Precision	Recall	F1-score	ROC-AUC
Logistic Regression	0.726	0.722	0.699	0.711	0.797
Random Forest	0.725	0.706	0.733	0.719	0.807
XGBoost	0.726	0.708	0.731	0.720	0.801

Table 4: How each model performed on the 1,238 customers in the test set, which none of the models saw during training.

Each column answers a different question. **Accuracy** is the share of all 1,238 customers classified correctly. **Precision** asks: of everyone flagged as a likely churner, what fraction actually churned (low precision means wasted retention offers). **Recall** asks the opposite: of everyone who actually churned, what fraction did we catch (low recall means real churners slip away unnoticed). **F1** balances the two, and **ROC-AUC** is the most complete measure, grading the model’s ranking of churners above non-churners across every possible cutoff rather than just the 50% default.

All three models land in a fairly tight band: accuracy around 72–73%, ROC-AUC between 0.797 and 0.807 solid and well above a random guess (0.5), though not spectacular, since the models only see behavioral RFM-style signals and nothing about product categories or marketing touches. The one gap worth noting: Random Forest trades a little precision (0.706, the lowest) for the best recall (0.733, the highest) exactly the trade a churn model should make when missing a real churner costs more than one extra false alarm (Section 5.9).

The table above only shows how well each model catches *churners* (class 1). Breaking precision, recall, and F1 down by class as well shows the mirror picture how well each model recognizes customers who are *staying* (class 0):

Model	Class	Precision	Recall	F1-score
Logistic Regression	Retained (0)	0.730	0.751	0.740
	Churned (1)	0.722	0.699	0.711
Random Forest	Retained (0)	0.744	0.717	0.730
	Churned (1)	0.706	0.733	0.719
XGBoost	Retained (0)	0.744	0.722	0.732
	Churned (1)	0.709	0.731	0.720

Table 5: Precision, recall, and F1-score per model, broken down by class, at the default 0.5 threshold.

All three models score higher on Retained than Churned across every metric – a loyal customer’s pattern is fairly consistent, while “about to churn” covers a wider range of behavior, making that class inherently noisier to predict. Logistic Regression is the most “precision-heavy” on churn (0.722, the highest, but the lowest recall at 0.699), while the tree models are more “recall-heavy,” catching more real churners at a small precision cost. The gap between classes is also smaller for the tree models (F1 gap of 0.011–0.012 vs. Logistic Regression’s 0.029), meaning they treat both classes a little more evenly.

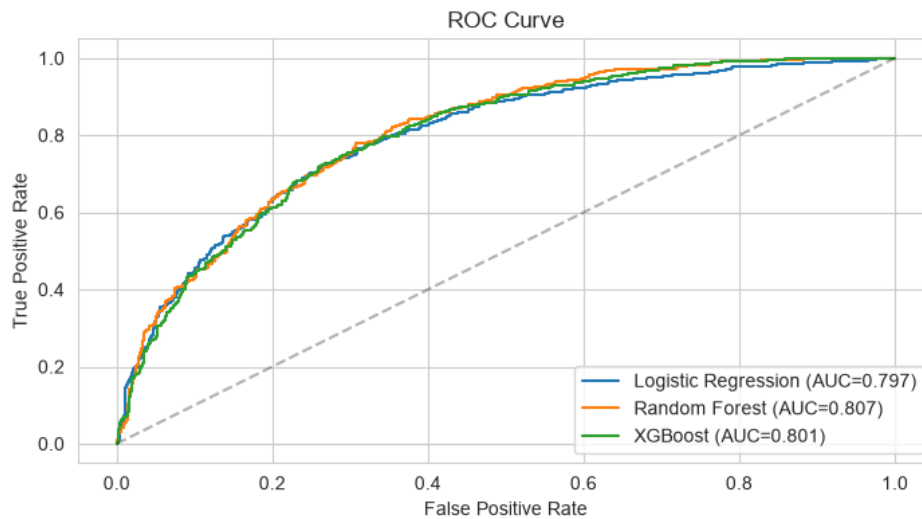


Figure 14: ROC curves for the three models. The closer a curve hugs the top-left corner, the better that model separates churners from non-churners. All three sit well above the diagonal “random guess” line, and track each other closely across almost the whole range.

The scores are close, which tells us the RFM-style features already capture most of what matters, and the relationship between behavior and churn is not wildly non-linear.

5.7 Checking for Overfitting

We also checked whether any model was just memorizing the training data, by comparing performance on data it trained on versus data it had never seen, shown below alongside the same five metrics side by side.

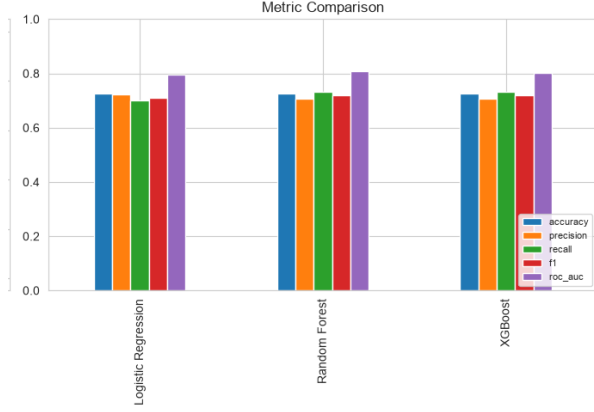


Figure 15: Accuracy, precision, recall, F1, and ROC-AUC for each model, side by side. All three models land in a similar range; no single one wins by a landslide.

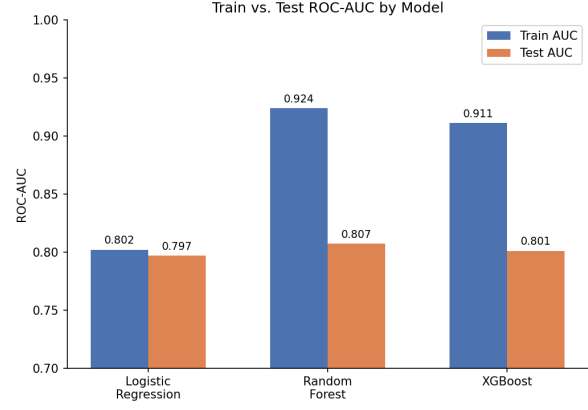


Figure 16: Train versus test performance. Logistic Regression barely changes between the two, so it is very stable. The tree models do noticeably better on training data than on new data, a sign of mild overfitting.

Logistic Regression’s train and test scores were nearly identical, a gap of just 0.005, so it generalizes cleanly. The tree-based models scored noticeably higher on training data than on the test set, with gaps of 0.11 to 0.12, a normal side effect of how flexible trees are, but worth watching if this model goes into production.

5.8 Choosing the Best Model

By ROC-AUC, **Random Forest** came out slightly ahead (0.807), correctly identifying 73% of actual churners (recall) while being right 71% of the time it flagged someone as churned (precision), a reasonably balanced result for a first version of this model.

5.9 Confusion Matrix Analysis

A single accuracy number can hide more than it reveals, so before trusting any of these models we broke each one’s predictions down into a **confusion matrix**: a 2×2 table sorting every test-set prediction into four buckets (true/false positive, true/false negative), comparing what the model *predicted* against what *actually* happened. Precision, recall, and F1 are all computed directly from these four counts:

$$\text{Precision} = \frac{TP}{TP + FP} \quad (\text{of everyone flagged as churning, how many really were}) \quad (17)$$

$$\text{Recall} = \frac{TP}{TP + FN} \quad (\text{of everyone who really churned, how many we caught}) \quad (18)$$

$$F_1 = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \quad (\text{a single number balancing the two}) \quad (19)$$

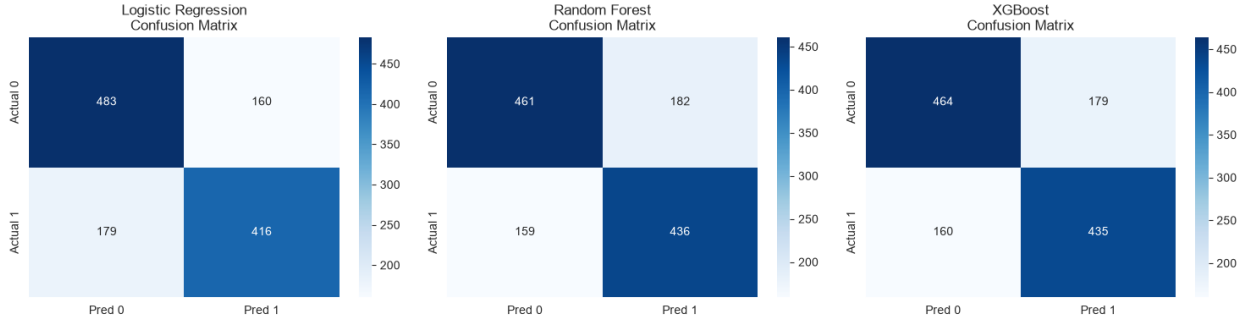


Figure 17: Confusion matrices for all three models on the 1,238 customers in the test set (643 retained, 595 churned). Each row is what actually happened; each column is what the model predicted.

Model	TN	FP	FN	TP	Missed churners	False alarms
Logistic Regression	483	160	179	416	179 of 595 (30.1%)	160 of 643 (24.9%)
Random Forest	461	182	159	436	159 of 595 (26.7%)	182 of 643 (28.3%)
XGBoost	464	179	160	435	160 of 595 (26.9%)	179 of 643 (27.8%)

Table 6: Confusion matrix counts by model, with each error type expressed as a share of its own class.

A missed churner costs more than a false alarm here (a lost customer vs. an unnecessary discount email), so **recall matters more than precision**. Random Forest reflects that best, missing the fewest actual churners (159 of 595) at the cost of a few more false alarms (182 of 643); Logistic Regression is the more conservative opposite, missing more churners (179) but raising fewer false alarms (160). This cost trade-off, not just the ROC-AUC edge from Section 5.7, is why Random Forest is the model this CRM team should actually use.

6 Model Interpretation: What Predicts Churn

A model that only outputs a risk score is not very useful to a CRM team on its own; what they need to know is *why* a customer is flagged, so the reason can guide the action taken. Across all three models, the simple one and the two complex ones, the same two signals kept coming out on top: **how recently someone last bought something**, and **how often they buy**.

Feature	Logistic Regression		Random Forest Importance	XGBoost Importance
	Coefficient	Odds Ratio		
Recency	0.878	2.405	0.232	0.209
Frequency	−0.588	0.555	0.092	0.184
PurchaseRate	−0.111	0.895	0.181	0.156
Monetary	−0.076	0.927	0.144	0.096
TotalItems	0.016	1.016	0.129	0.117
UniqueProducts	−0.537	0.584	0.061	0.066
AvgOrderValue	−0.055	0.946	0.058	0.059
Tenure	−0.164	0.849	0.052	0.058
AvgItemsPerOrder	0.045	1.046	0.051	0.054

Table 7: The contribution of each of the nine features to the churn prediction in each of the three models. Gini and gain-based, respectively, Random Forest and XGBoost importances add up to 1.0 in their respective columns.

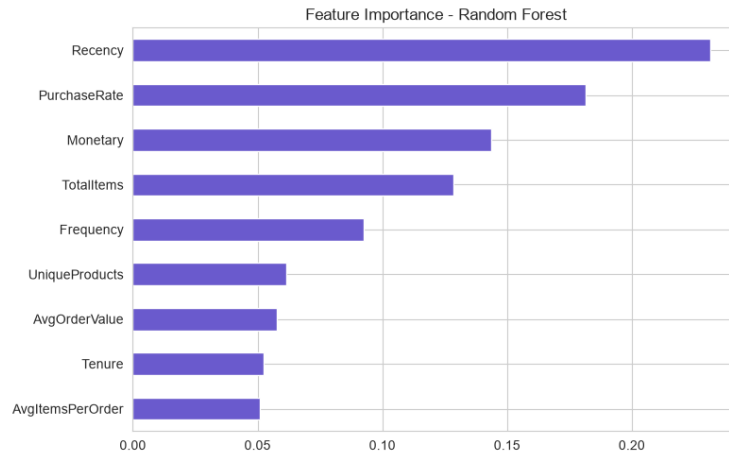


Figure 18: Feature importance for the winning model (Random Forest) on its own. Recency alone accounts for roughly 23% of the model’s total decision-making weight.

For the CRM team, this means the churn flag is not a black box: a customer is flagged mainly because they have gone quiet and slowed down how often they buy, the same two signals that already drive the segmentation in Section 4. That overlap is reassuring, it means the churn model is not picking up on some strange artifact of the data, but is reinforcing the same story the segments already tell, just with a forward-looking, customer-by-customer score instead of a static label.

7 Discussion: Limitations, Ethics, and Future Work

7.1 Limitations

A few things are worth being upfront about. First, the UK makes up 83.8% of revenue in this dataset (Section 2.5), so both the segments and the churn model are, in practice, mostly describing UK shopping behavior; results may not transfer cleanly to the smaller international markets. Second, the RFM features that drive both the clustering and the churn model are purely behavioral. We have no data on product category preferences, marketing touches (emails, ads, promotions received), customer service interactions, or demographics, any of which could sharpen both the segments and the predictions. Third, the tree-based churn models (Random Forest and XGBoost) showed a noticeable train/test performance gap (Section 5.7), a sign of mild overfitting that more aggressive regularization or a larger training window could likely reduce. Finally, the churn definition itself no purchase within 180 days is a reasonable business rule but a somewhat arbitrary one; a shorter or longer window could shift both the churn rate and which customers get flagged.

7.2 An Ethical or Bias Risk Worth Naming

The clearest risk here is treating the segments and churn flags as a reason to under-serve customers rather than as a prioritization tool. A “Lost/Let Go” or low-priority churn-risk label is a statement about *past purchasing behavior*, not about a customer’s worth as a person, and it can be wrong: a customer who moved countries, had a one-off large purchase for a wedding, or simply shops seasonally could easily be mislabeled by a purely RFM-based rule. If this model were used to silently downgrade the level of service (support response times, return policies, etc.) a customer receives, rather than just to prioritize proactive marketing outreach, that would cross from reasonable business prioritization into something that could unfairly and invisibly harm real customers. We recommend that any customer-facing use of these segments be limited to *opt-in* marketing prioritization, not core service quality.

7.3 What We Would Do With More Time

With more time, we would: (1) bring in marketing-touch data (which customers actually received which campaigns) to measure whether the recommended interventions in Section 4.3 actually move customers between segments, rather than just assuming they would; (2) tune the tree-based models

more aggressively to close the overfitting gap noted above; (3) test alternative churn-window lengths (e.g. 90 or 365 days) to see how sensitive the results are to that choice; and (4) build a small internal dashboard so the CRM team can look up any customer's segment and churn risk directly, instead of working from a static report.

8 Conclusion & Recommendations

The results from customer segmentation and churn prediction provide the business with a clear understanding of its customers. The “At Risk” customers had an average of 394 days since their last purchase, showing that they are already becoming inactive. The churn model can help identify these customers earlier, giving the business more time to take action.

The results also show that the features used were more important than the choice of machine learning algorithm. All three churn models achieved similar ROC-AUC results. Recency was the strongest indicator of churn, followed by purchase frequency. Although Random Forest performed best overall, with 72.5% accuracy, 70.6% precision, 73.3% recall, and a ROC-AUC of 0.807, Logistic Regression showed more consistent performance between training and testing data.

Customer segmentation also showed that the VIP group, although representing only 15.4% of customers, generated over 70% of total revenue, while the One-Time Buyer and Lost/Let Go segments together made up over 41% of customers but under 5% of revenue. This highlights the importance of treating customers differently according to their value and behaviour, rather than spending marketing budget evenly across a base that is this lopsided.

Overall, segmentation helps the business understand **who its most valuable customers are today**, while churn prediction helps identify **who may leave tomorrow**. Together, these approaches can help the CRM team focus its efforts, protect valuable customers, and reduce customer losses.

References

- [1] Chen, D. & UCI Machine Learning Repository. *Online Retail II Data Set*. <https://archive.ics.uci.edu/dataset/502/online+retail+ii>
- [2] Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
- [3] Chen, T. & Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*.

Appendix: How the Customer Explorer App Works

As a bonus deliverable, we built **Customer Explorer**, a Streamlit app that lets the CRM team look up any customer and instantly see their segment, spending pattern, and churn risk. The app does none of the heavy computation itself: a separate pipeline script (`scripts/run_pipeline.py`) runs the cleaning, RFM scoring, clustering, and churn-model training once and saves the results as CSV files and a trained model, which the app simply reads and displays. This keeps the app lightweight and fast to open, since it only needs Streamlit, pandas, numpy, and plotly installed, not the modelling libraries (scikit-learn, XGBoost, mlxtend).

The app is organized into five pages, accessible from a sidebar (Customer Explorer, Segment Overview, Churn Model, Market Basket, and About). From there, a CRM user can search for any customer by ID or filter by segment to see their RFM values, RFM score, and churn probability at a glance, alongside interactive PCA, revenue, and churn-probability charts, plus product association rules from the market-basket analysis. Extra detail, such as full purchase history, stays tucked into expandable sections so the main screen stays uncluttered, and the customer list or the at-risk list can be exported directly as a CSV. If the underlying pipeline has not been run yet, the app shows clear setup instructions instead of failing.